

DATA ANALYSIS OUTPUT ON SUPERSTORE DATA

Subqueries: Subqueries are queries written inside another SQL query. They help filter, compare, or calculate values based on results returned by the inner query. They can be used in SELECT, WHERE, or FROM clauses.

For above average sales:

This query finds sales records where the sales amount is greater than the average sales amount. It is commonly used to identify high-performing orders or customers. An aggregate function is usually used in the subquery.

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Product ID	Sales	Quantity	Discount	Profit
▶	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-BO-10001798	261.96	2	0	41.9136
	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-CH-10000454	731.94	3	0	219.582
	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	FUR-TA-10000577	957.5775	5	0.45	-383.031
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	TEC-PH-10002275	907.152	6	0.2	90.7152
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	FUR-TA-10001539	1706.184	9	0.2	85.3092
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	TEC-PH-10002033	911.424	4	0.2	68.3568
	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	OFF-BI-10003656	407.976	3	0.2	132.5922
	CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19075	OFF-ST-10004186	665.88	6	0	13.3176
	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	FUR-TA-10000577	1044.63	3	0	240.2649
	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	FUR-BO-10004834	3083.43	7	0.5	-1665.0522
	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	TEC-PH-10004977	1097.544	7	0.2	123.4737
	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	FUR-BO-10002545	532.3992	3	0.32	-46.9764

orders 1 x

For highest order per customer

This query identifies the order with the highest value for each customer. It helps analyze customer purchasing behavior and top transactions. MAX() or window functions are commonly used.

Result Grid		Filter Rows:		Export:		Wrap Cell Content:			
Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Product ID	Sales	Quantity	Discount	Profit
CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-CH-10000454	731.94	3	0	219.582
US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	FUR-TA-10000577	957.5775	5	0.45	-383.031
CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	FUR-TA-10001539	1706.184	9	0.2	85.3092
CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	FUR-TA-10000577	1044.63	3	0	240.2649
US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	FUR-BO-10004834	3083.43	7	0.5	-1665.0522
CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	TEC-PH-10004977	1097.544	7	0.2	123.4737
CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	FUR-BO-10002545	532.3992	3	0.32	-46.9764
CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	TEC-PH-10002447	1029.95	5	0	298.6855
CA-2016-111682	6/17/2016	6/18/2016	First Class	TB-21055	FUR-CH-10003968	319.41	5	0.1	7.098
CA-2014-106376	12/5/2014	12/10/2014	Standard Class	BS-11590	OFF-AR-10002671	1113.024	8	0.2	111.3024
CA-2015-129476	10/15/2015	10/20/2015	Standard Class	PA-19060	TEC-AC-100002671	67.992	5	0.2	67.992

CTE stands for Common Table Expression.

It is a temporary result set that exists only during the execution of a query and is created using the WITH clause.

Total sales generated by each customer

This analysis calculates the total revenue contributed by every customer. GROUP BY and SUM() functions are typically used. It helps identify valuable customers for the business.

Result Grid		Filter Rows:
	Customer_ID	Total_Sales
▶	SM-20320	25043.05
	TC-20980	19017.847999999998
	RB-19360	15117.339
	TA-21385	14595.62
	AB-10105	14355.610999999997
	SC-20095	14142.333999999999
	KL-16645	14071.917
	HL-15040	12873.297999999999
	SE-20110	12209.438000000002
	CC-12370	12129.072
	TS-21370	11885.871
	GT-14710	11820.119999999997

Total profit generated by each customer

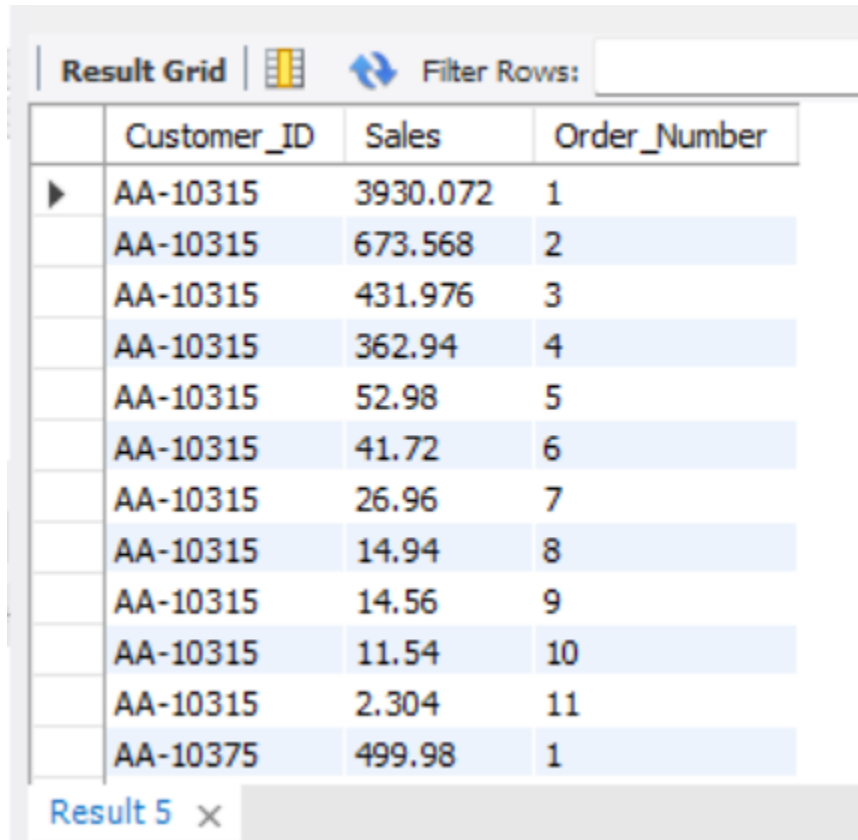
This query calculates the profit earned from each customer by summing profit values. It helps measure customer profitability and supports business decision-making.

Result Grid		Filter Rows:
	Customer_ID	Total_Profit
▶	TC-20980	8964.482600000001
	RB-19360	6976.0959
	SC-20095	5757.411899999999
	HL-15040	5622.4292000000005
	AB-10105	5438.9075
	TA-21385	4703.788299999999
	CM-12385	3899.8903999999998
	KD-16495	3038.6254
	AR-10540	2884.6207999999997
	DR-12940	2869.0760000000005
	TB-21400	2790.7099000000003
	NM-18445	2751.684899999999

Result 4 x

Window functions: A window function performs calculations across a set of related rows while retaining each individual row in the result set.

1. **ROW_NUMBER()** assigns a unique sequential number to each row within a partition. It is useful for ranking, pagination, and finding duplicate records. Numbers are assigned without ties.



	Customer_ID	Sales	Order_Number
▶	AA-10315	3930.072	1
	AA-10315	673.568	2
	AA-10315	431.976	3
	AA-10315	362.94	4
	AA-10315	52.98	5
	AA-10315	41.72	6
	AA-10315	26.96	7
	AA-10315	14.94	8
	AA-10315	14.56	9
	AA-10315	11.54	10
	AA-10315	2.304	11
	AA-10375	499.98	1

Result 5 ×

2. **RANK()** assigns ranks to rows based on a specified order. Rows with the same value receive the same rank, and gaps appear in the ranking sequence after ties.

Result Grid   Filter Rows: Export			
	Customer_ID	Total_Sales	Customer_Rank
▶	SM-20320	25043.05	1
	TC-20980	19017.847999999998	2
	RB-19360	15117.339	3
	TA-21385	14595.62	4
	AB-10105	14355.610999999997	5
	SC-20095	14142.333999999999	6
	KL-16645	14071.917	7
	HL-15040	12873.297999999999	8
	SE-20110	12209.438000000002	9
	CC-12370	12129.072	10
	TS-21370	11885.871	11
	GT-14710	11820.119999999997	12
	BM-11140	11609.900000000001	13
	SV-20365	11420.645999999997	14
	CI-12010	11070.742	15

Result 6 × 

.....

3. **DENSE_RANK()** also assigns ranks based on ordering. Rows with equal values receive the same rank, but unlike RANK(), no gaps are left in the ranking sequence.

The screenshot shows a 'Result Grid' window with a table of customer sales data. The table has four columns: 'Customer_ID', 'Total_Sales', and 'Dense_Rank_Value'. The data is sorted by total sales in descending order. The ranks are assigned sequentially from 1 to 15, with no gaps, indicating the use of DENSE_RANK(). The interface includes a 'Filter Rows' field and an 'Export' button.

Customer_ID	Total_Sales	Dense_Rank_Value
SM-20320	25043.05	1
TC-20980	19017.847999999998	2
RB-19360	15117.339	3
TA-21385	14595.62	4
AB-10105	14355.610999999997	5
SC-20095	14142.333999999999	6
KL-16645	14071.917	7
HL-15040	12873.297999999999	8
SE-20110	12209.438000000002	9
CC-12370	12129.072	10
TS-21370	11885.871	11
GT-14710	11820.119999999997	12
BM-11140	11609.900000000001	13
SV-20365	11420.645999999997	14
CT-12010	11070.742	15

Final JOIN + CTE + WINDOW FUNCTION

This combines JOINS, CTEs, and window functions to create a comprehensive report. The result shows customer details, total sales, and ranking. It demonstrates how multiple SQL concepts work together in real-world analysis.

The screenshot shows a 'Result Grid' window displaying a comprehensive report. The table has five columns: 'Customer ID', 'Customer Name', 'Total_Sales', and 'Sales_Rank'. The data is sorted by total sales in descending order. The ranks are assigned sequentially from 1 to 14, with no gaps, indicating the use of DENSE_RANK(). The interface includes a 'Filter Rows' field and an 'Export' button.

Customer ID	Customer Name	Total_Sales	Sales_Rank
SM-20320	Sean Miller	25043.05	1
TC-20980	Tamara Chand	19017.847999999998	2
RB-19360	Raymond Buch	15117.339	3
TA-21385	Tom Ashbrook	14595.62	4
AB-10105	Adrian Barton	14355.610999999997	5
SC-20095	Sanjit Chand	14142.333999999999	6
KL-16645	Ken Lonsdale	14071.917	7
HL-15040	Hunter Lopez	12873.297999999999	8
SE-20110	Sanjit Engle	12209.438000000002	9
CC-12370	Christopher Conant	12129.072	10
TS-21370	Todd Sumrall	11885.871	11
GT-14710	Greg Tran	11820.119999999997	12
BM-11140	Becky Martin	11609.900000000001	13
SV-20365	Seth Vernon	11420.645999999997	14